COMP90041
Programming and Software Development

Lecture 8 - Modularity & Interfaces

Danny Xu

The University of Melbourne acknowledges the Traditional Owners of the unceded land on which we work, learn and live: the Wurundjeri Woi-wurrung and Bunurong peoples (Burnley, Fishermans Bend, Parkville, Southbank and Werribee campuses), the Yorta Yorta Nation (Dookie and Shepparton campuses), and the Dja Dja Wurrung people (Creswick campus).

The University also acknowledges and is grateful to the Traditional Owners, Elders and Knowledge Holders of all Indigenous nations and clans who have been instrumental in our reconciliation journey.

We recognise the unique place held by Aboriginal and Torres Strait Islander peoples as the original owners and custodians of the lands and waterways across the Australian continent, with histories of continuous connection dating back more than 60,000 years. We also acknowledge their enduring cultural practices of caring for Country.

We pay respect to Elders past, present and future, and acknowledge the importance of Indigenous knowledge in the Academy. As a community of researchers, teachers, professional staff and students we are privileged to work and learn every day with Indigenous colleagues and partners.

THE UNIVERSITY OF MELBOURNE

Overview

In this part of the lecture, you will learn about:

- **Modularity**
- **Unified Modelling Language (UML)**
- **Interfaces**

Modularity

What and Why?

- A module is the basic unit of decomposition of our systems, e.g., – packages
- Modular design and programming is designing or constructing software based on several modules
  - Hierarchy of smaller modules like packages
  - One package has several related classes
  - One class has related methods.

Modular design criteria

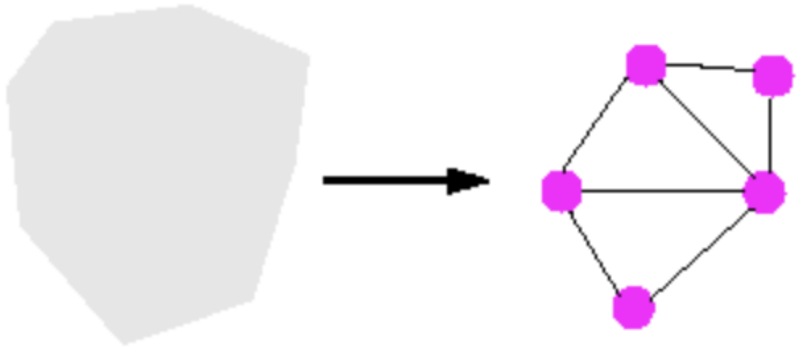
Five Criteria for Modular design -

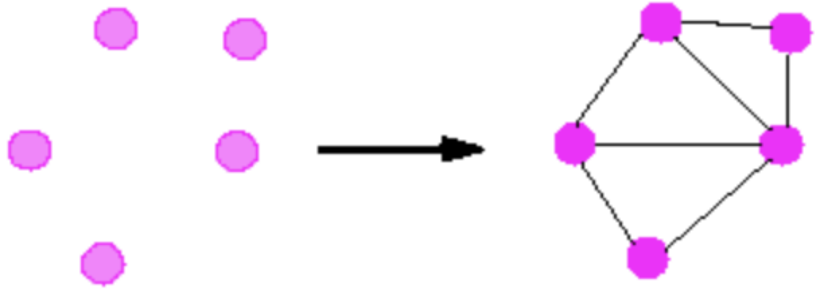
- Decomposability
- Composability
- Understandability
- Continuity
- Protection

Decomposability vs Composability

- decompose a problem into a small number of less complex sub-problems
- independent enough to allow further work to proceed separately on each of them

- components that can then be freely combined with each other to produce new systems,





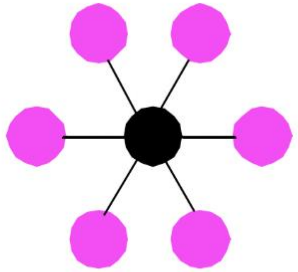
Other Design Criteria

- Understandability - human reader can understand each module without having to know/understand the other module.
- Continuity - small change in a problem specification will trigger a change of just one module, or a small number of modules, not entire codebase.
- Protection - abnormal condition occurring at run time in one module will remain confined to that module or at worst will only propagate to a few neighbouring modules.

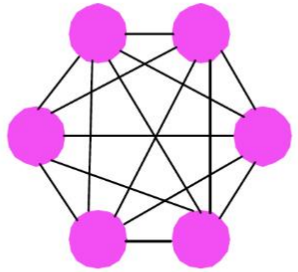
Module Design rules

1. Direct Mapping
   - Map problems to real world entities
   - Map structure of the problem to the solution
   - Modular Criteria: Continuity, Decomposability

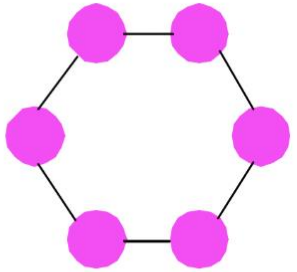
2. Few Interfaces
   - overall number of communication channels or interfaces between modules should be restricted
   - "interface" refers to things like the set of public methods, variables, etc.
   - Modular criteria: Continuity, Protection, Understandability, Composability and Decomposability



(A)
n-1
connections



(B)
n * (n - 1) /2
connections



(C)
n
connections

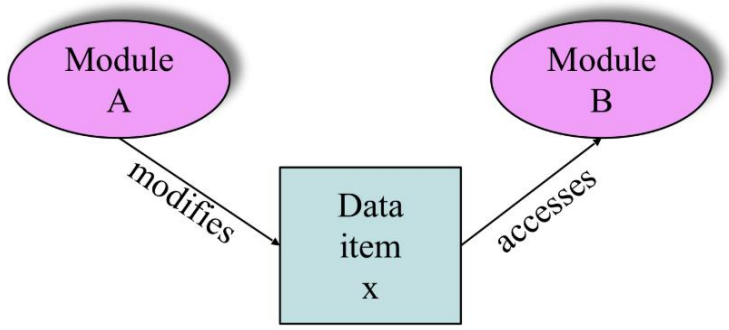
Modular Design rules

3. Small Interfaces

- Interfaces should be small in size exchanging minimal information
- Fewer public method, methods should not be dependent on other methods
- Simple is better or "as simple as possible but no simpler"
- Okay to have larger interfaces sometimes but stick to minimal
- Modular criteria: Continuity, Protection.

4. Explicit Interfaces

- Whenever two modules A and B communicate, this must be obvious from the text of A or B or both.
- Modular Criteria: Decomposability, Composability, Continuity and Understandability.



Modular Design rules

5. Information hiding
    - Must select a subset of the module's properties to be made visible
    - Modular Criteria: Continuity

Unified Modelling Language (UML)

Unified Modelling Language

Modularity is important. How to describe them?

- The Unified modelling language (UML) is a graphical representation of modules and their interactions.
- UML covers a lot of different depictions. Some of the famous ones are –
  - Domain Model diagrams – maps the problem statement into real-world entities.
  - Class Diagrams – represents the solutions i.e. classes and associations between them
  - Sequence diagrams – how the data flows between the entities.
  - And many many more...

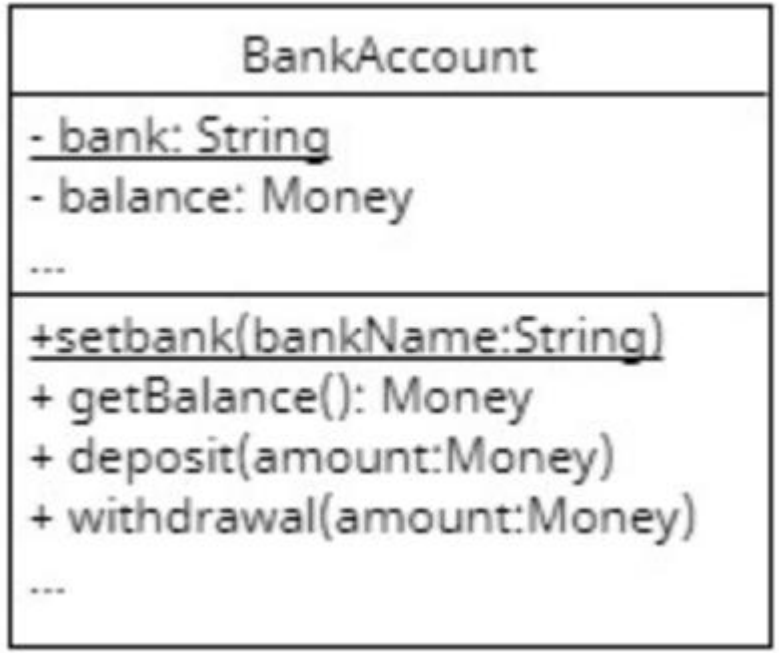
UML entities

- Classes
- Packages
- Inheritance
- Associations
- Aggregations

UML Entities: Classes

Class is divided into three sections

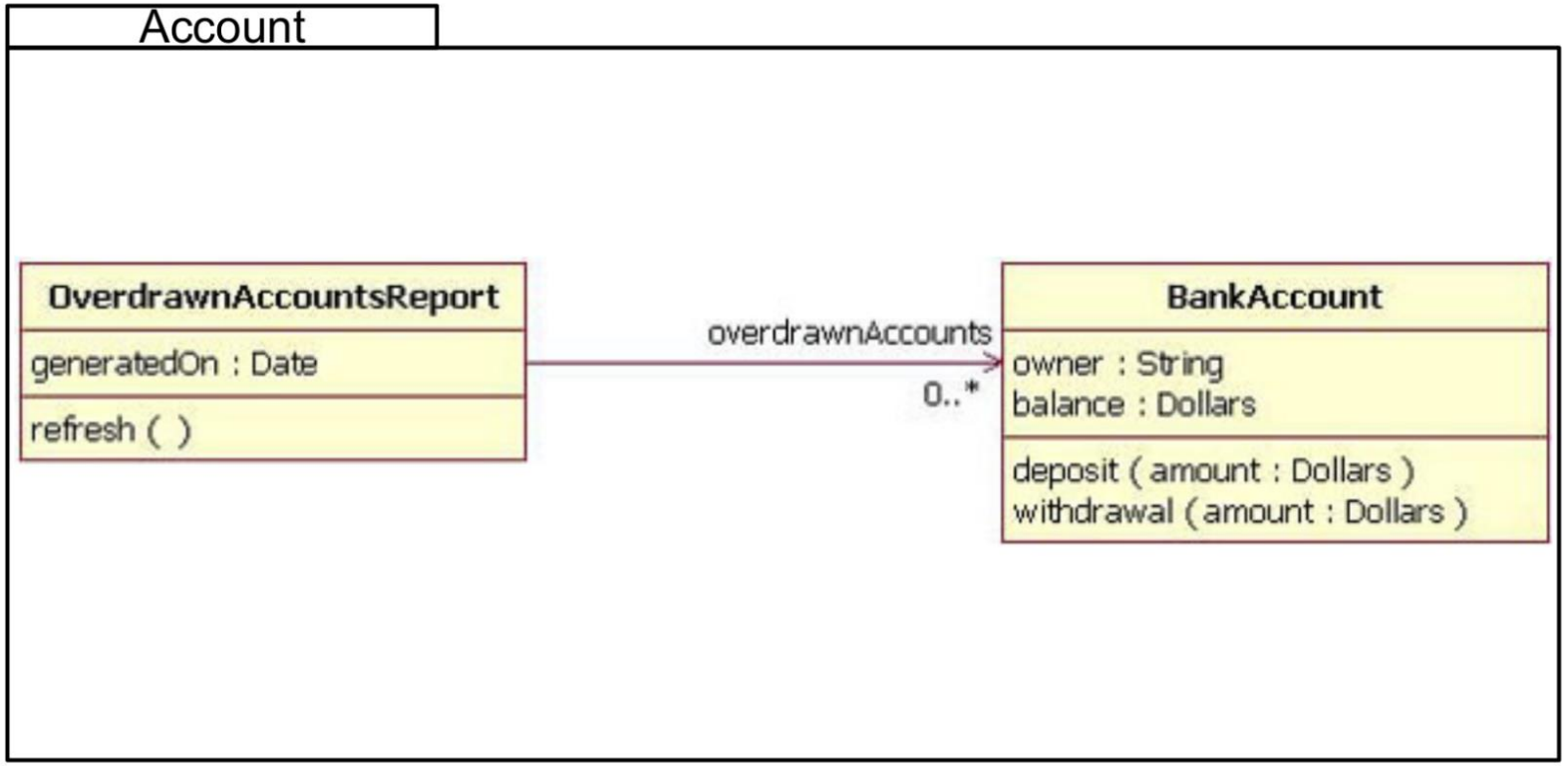
- Top part – Class Name
- Middle part – Attributes
  - Name
  - Access modifiers
  - Data types
- Bottom – methods
  - Method name
  - Arguments
  - Return type
- Access modifiers
  - + public
  - - private
  - # protected
  - ~ package/default



- No need to put all unnecessary private details
- Missing members notified by three dots
- Underlined for static members

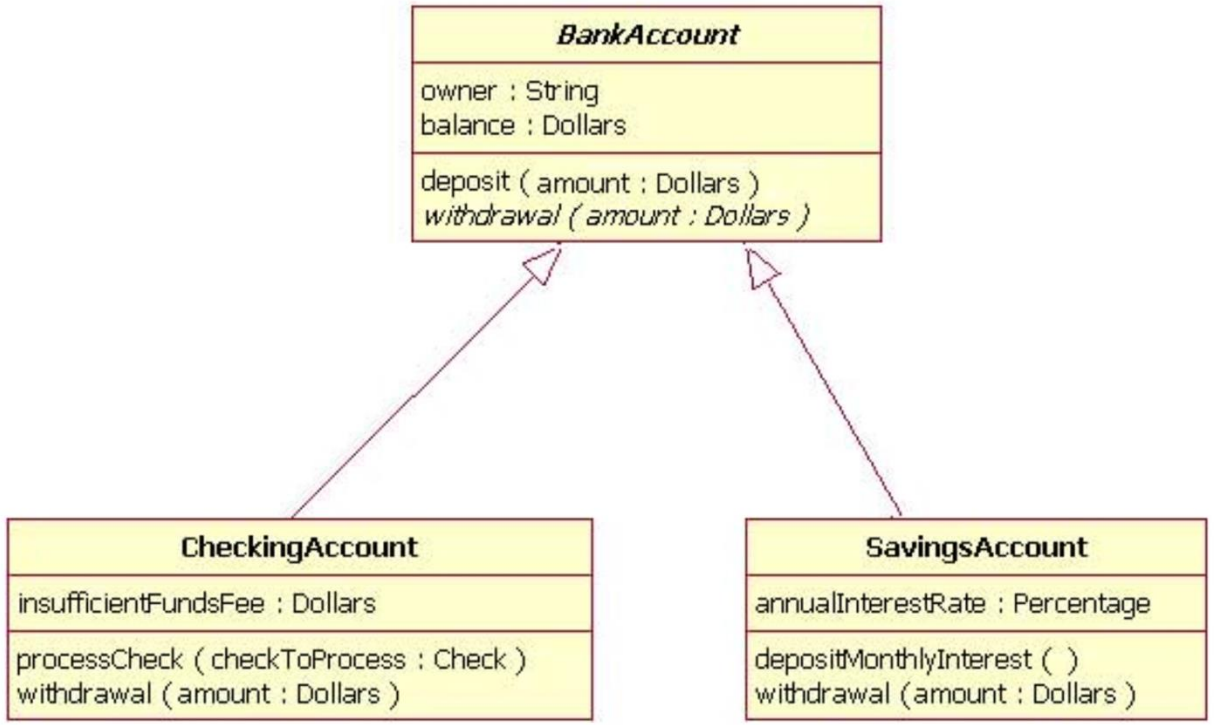
UML Entities: Packages

- Wrap multiple classes into a big rectangular box with a name tab. The name is the package name



UML Entities: Inheritance

- Inheritance is shown by an arrow with an open head leading from the child class to the parent class



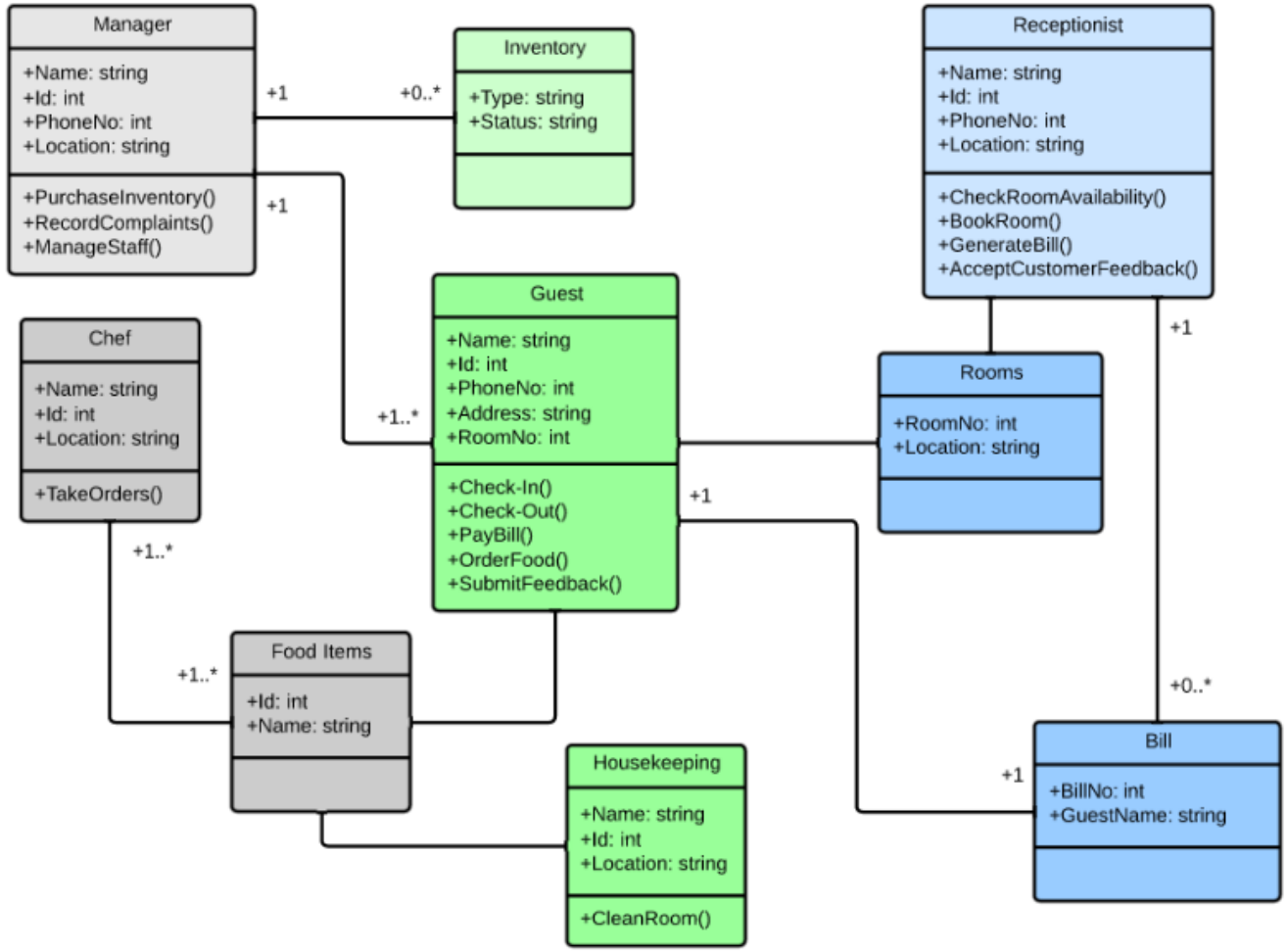
UML Entities: Association

- Something (unspecified) linking two classes
  - One object uses or interacts with another object via method calls, object ref
  - Classes implementing interfaces

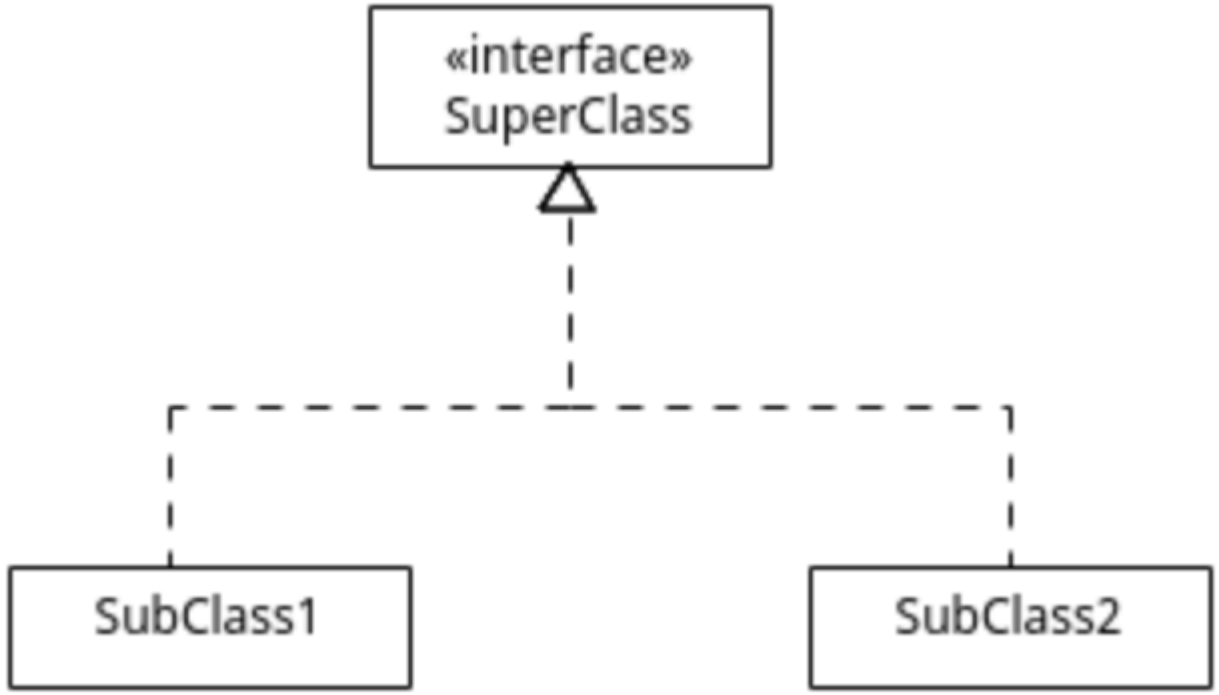
- Associations are directional
  - Nondirectional – no arrow, just a straight line joining two classes
  - Unidirectional – arrows pointing from one to other
  - Bidirectional – arrows pointing towards both classes

- Associations has multiplicity
  - 1 – exactly one
  - n – exactly n, where n could be any positive number
  - 1..1 - one to one
  - 1..* - one to many
  - 0..* - zero to many

UML Entities: Associations



UML Entities: Interface Associations

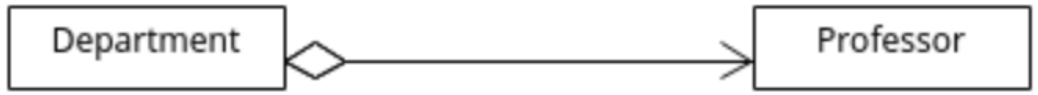


UML Entities: Aggregation

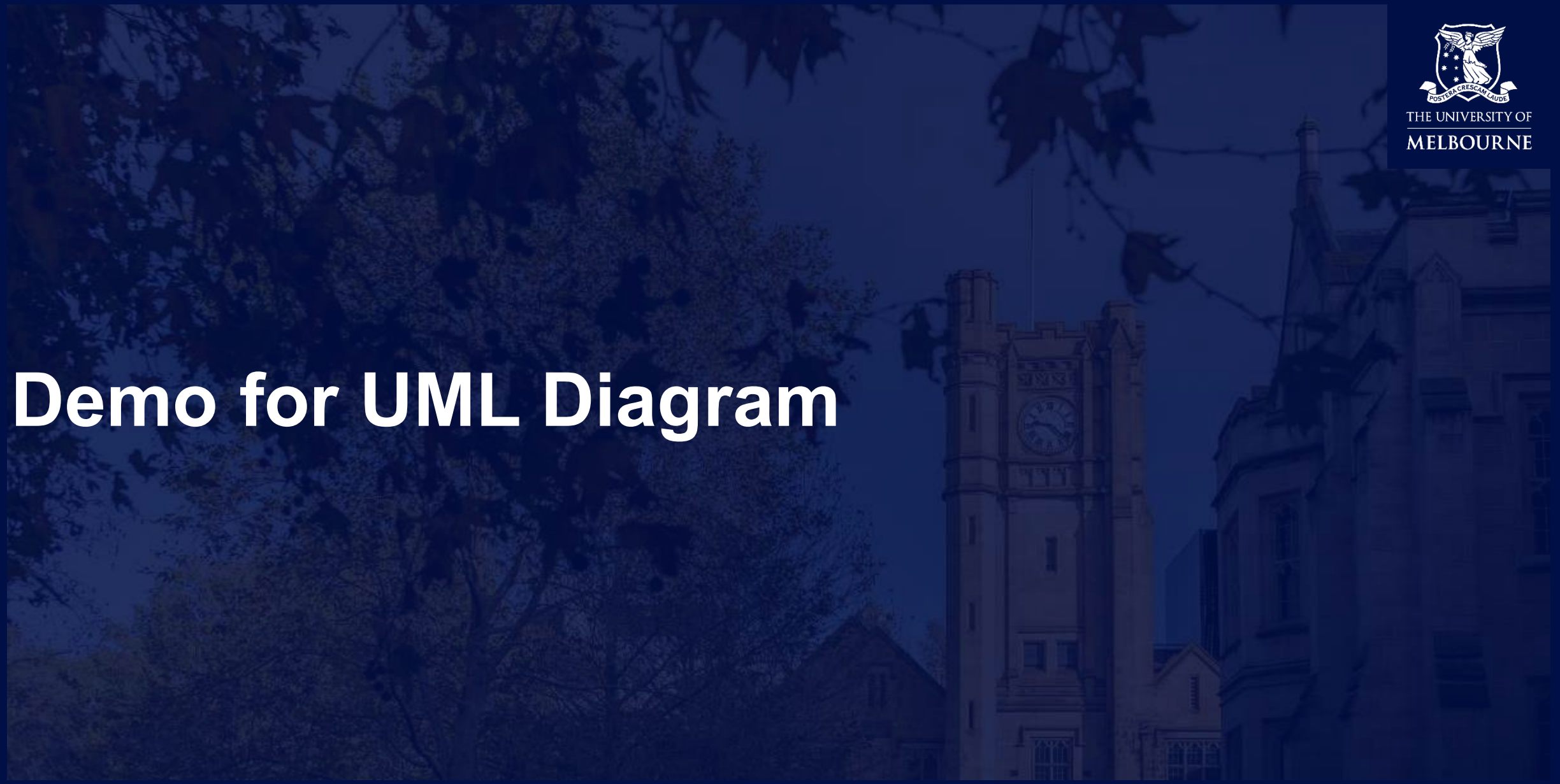
- Aggregation can be shared or composed
- Composition is where an object is composed of other objects. The other object cannot exist without the first one.

UML Composition. University consists of Departments.

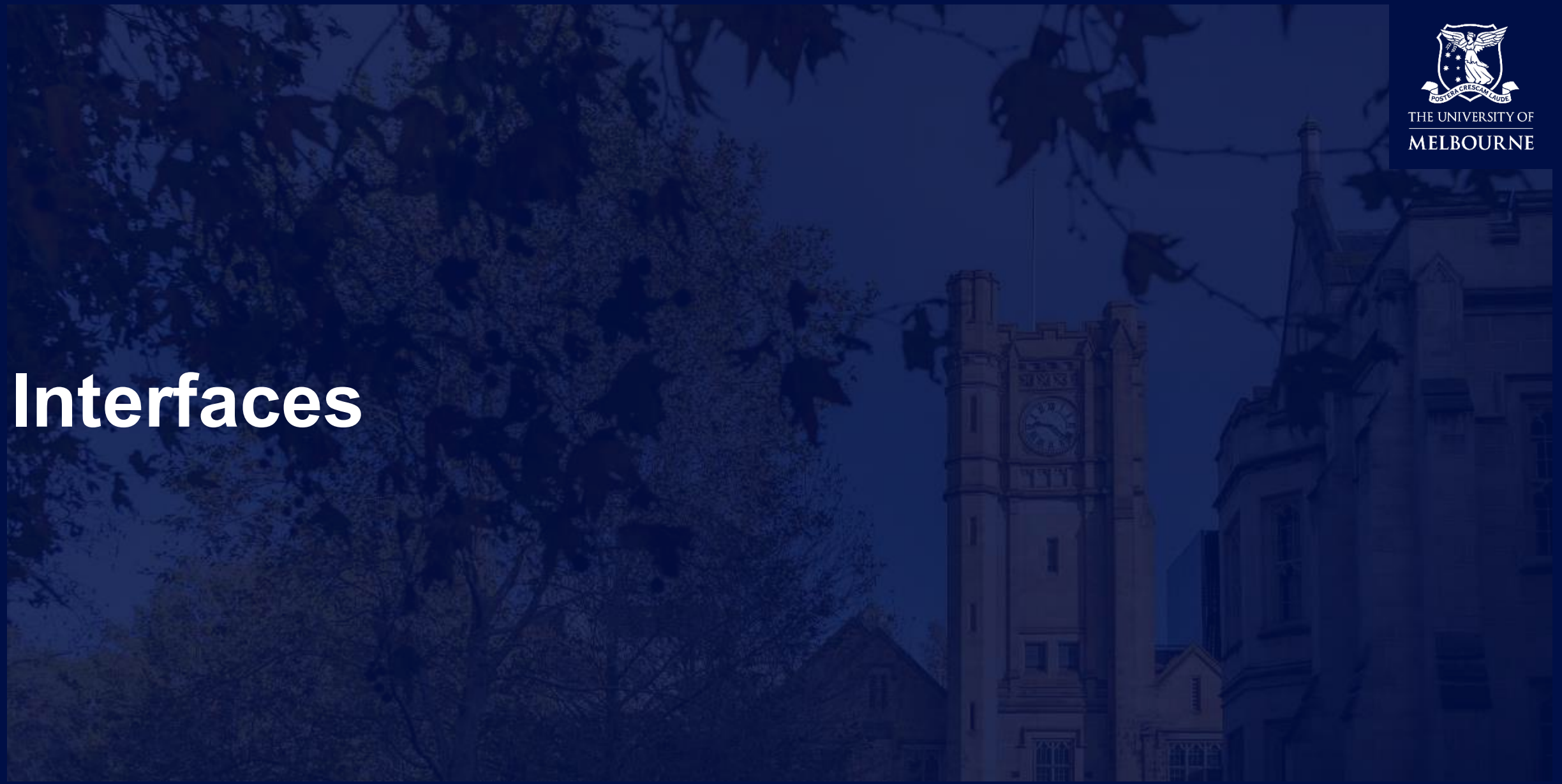
- Shared aggregation is where an object contains reference to another one. The individual objects can exist independently.



UML aggregation (shared). Department contains professors.
The professors may belong to more than one departments.



Demo for UML Diagram



Interfaces

THE UNIVERSITY OF MELBOURNE

Interfaces

- Interfaces are just set of methods that a class promises to implement.
- Interface is not a class. They are a different type.
- An interface specifies method definitions but not implementations
  - Method names
  - Arguments
  - Return types
  - No implementation, no {}, ends with a semicolon
- Can we define variables in interfaces?
  - No. Only constants. `final and static` keywords are implicit here.
  - Define constants if the implemented class wants to use the constants.
  - Have an interface just to define constants – Bad Practice!
- Access modifier – methods & constants are always public even when not explicitly mentioned. private or protected not allowed

Implementing Interfaces

- A class implements interface. Use the keyword implements
- A class cannot change the method definitions defined in the interface
- An abstract class can implement some or all the methods of the interface it is implementing.
- A class must implement all the methods in interface or declare the class "abstract"
- If a parent class implements the methods from interfaces, the child class automatically inherits them and can override them.

Example

No explicit public static, or final

```
public interface Calculable{
      double PI = 3.14;

     double surfaceArea();
     public double volume();
}
```

Implements some but not all methods of interface

```
public abstract class CircularBase implements
Calculable{
      double radius;
      double surfaceArea() {
         return PI * this.radius * this.radius;
      }
}
```

```
public class Cylinder extends CircularBase{
   double height;

   double surfaceArea() {
     return 2 * super.surfaceArea() + 2 *
        PI* super.radius * this.height;
    }

   double volume() {
     return 1/3 * super.surfaceArea() *
       this.height;
   }
}
```

Overrides the methods from parent

```
public class Cuboid implements Calculable{
      double side;

      double surfaceArea() {
         return side * side;
      }

      double volume() {
            return side * side * side;
      }
}
```

Directly implements an interface

Abstract Classes vs Interfaces

- When to create abstract class vs when to create interfaces?
- Abstract classes
  - When classes have some common implementation but not all of them
  - Classes are related by a hierarchy
  - Example – Employees or Shapes like Circle, Cylinder, Cone, etc.

- Interfaces
  - When all the implementation differ and methods are generic features.
  - Classes are not related in anyway.
  - Example – Chargeable devices – mobile phones vs Cars vs Apple Watch. Classes are not related, do different things, have different implementations.

Derived Interfaces

- An interface that extends another interface(s) is called a Derived Interface.

```
public interface Payable {
    void pay(double amount);
}

public interface Refundable {
    void refund(double amount);
}

public interface Transactional extends Payable, Refundable {
    void printReceipt();
}
```

Comparable Interface

- Comparable interface is provided by java.lang automatically
- You can implement the method compareTo in your classes to compare if two objects are equal.
- The return type is int
  - 0 if two objects are equal
  - A negative number if the calling object "comes before" the parameter other
  - A positive number if the calling object "comes after" the parameter other
  - If the other object does not belong to same class, throw a `ClassCastException`

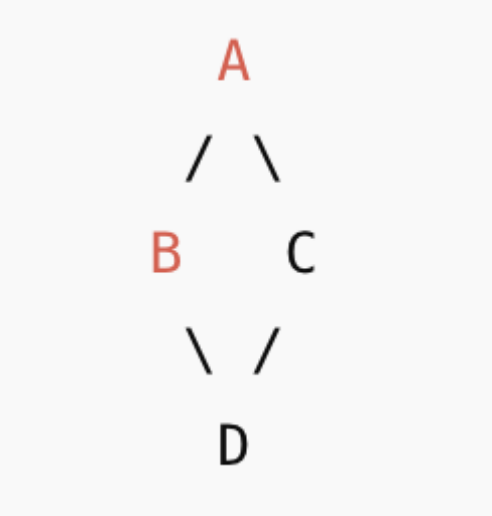
Comparable Interface: Example


```
public class Employee{
   int empId;
   String name;

   public int compareTo(Object otherEmp){
       if !(otherEmp instanceOf Employee)
           throw new ClassCastException();
       Employee other= (Employee)otherEmp;
       if(this.empId == other.empId && this.name.equalsIgnoreCase(other.name))
           return 0;
       else if(this.empId == other.empId){
           return this.name.compareTo(other.name);
       }else{
           return this.empId - other.empId;
       }

   }
}
```

Diamond Problem

- The **Diamond Problem** occurs in **multiple inheritance** when:
  - A class inherits from two classes
  - Those two classes inherit from a common superclass
  - Ambiguity arises: Which version of the method is inherited?

- Why is it called diamond?



Diamond Problem

```
class Parent1 {
    void fun() { System.out.println("Parent1"); }
}

class Parent2 {
    void fun() { System.out.println("Parent2"); }
}

// Inherting the Properties from Both the classes

class Test extends Parent1, Parent2 {

// main function
  public static void main(String[] args)
  {
      Test t = new Test();
      t.fun();  // which fun() to invoke Parent1.fun() or Parent2.fun()
  }
}
```

Diamond Problem

JAVA DOES NOT SUPPORT MULTIPLE INHERITANCE

```
class Parent1 {
    void fun() { System.out.println("Parent1"); }
}

class Parent2 {
    void fun() { System.out.println("Parent2"); }
}

// Inherting the Properties from Both the classes

class Test extends Parent1, Parent2 {

// main function
  public static void main(String[] args)
  {
      Test t = new Test();
      t.fun(); // which fun() to invoke Parent1.fun() or Parent2.fun()
  }
}
```

Multiple Interfaces

- Java does not support multiple inheritance.
- However, Java supports implementing Multiple Interfaces.
  - We usually won't write same functionality in two different interfaces.
- For implementing multiple interfaces use syntax
  - `classOne implements interfaceA, interfaceB, interfaceC`
  - You can add as many interfaces as you want.
- Java also allows to use inheritance and interface together. Use syntax
  - `classA extends classB implements interfaceC, interfaceD`

Implementing Multiple Interfaces

```
interface Parent1 {
    void fun();
}

interface Parent2 {
    void fun();
}

// class implementing multiple Interfaces
class Test implements Parent1, Parent2 {
    public void fun()
    {
        System.out.println("fun function");
    }

    public static void main(String[] args)
    {
        Test t = new Test();
        t.fun();
    }
}
```

Pitfalls : Inconsistent Interfaces

- When a class implements two interfaces:
  - One type of inconsistency will occur if the interfaces have constants with the same name, but with different values.
  - Another type of inconsistency will occur if the interfaces contain methods with the same name and signature but different return types

- If a class definition implements two inconsistent interfaces, then that is an error, and the class definition is illegal.

Live Coding

Live Coding – Interfaces!!

- One interface implemented by different classes
  - Printer, laptop, mobile phone implementing same interface Printable with different implementations.
- Multiple interfaces implemented by same class
  - Employee class implementing Payable, Workable.
  - Employee example with abstract class implementing interface.

Additional Reading Resources

- WALTER, S. Absolute Java, Global Edition. [Harlow]: Pearson, 2016. (Chapter 7 and 8)
- SCHILDT, H. Java: The Complete Reference, 12th Edition: McGraw-Hill, 2022 (Chapter 8 and 9)
- Object Oriented Programming Concepts (accessible on 14-02-2024) The Java Tutorials. Available at: https://docs.oracle.com/javase/tutorial/java/concepts/index.html
- Shvets, A. Dive Into DESIGN PATTERNS.

The University of Melbourne (Australian University)
PRV12150/CRICOS 00116K

See you next time!

The University of Melbourne (Australian University) PRV12150/CRICOS 00116K